Relatório de Performance

Portfólio Luis Fernandes — antes × depois das otimizações

Projeto: Portfólio profissional — projeto final do curso de Front-end (EBAC)

Autor: Luis Fernandes

Site: https://kmpc2013.github.io/ebac-frontend-projeto-final/

Medições: 25/10/2026 — Lighthouse 13.5.0

Indicador (Mobile)	Antes	Depois
Performance	35	85
Total Blocking Time	6481 ms	211 ms (−97%)
LCP	6,82 s	3,54 s (−48%)
Peso da página	1830,6 KB	396,1 KB (−78%)
Acessibilidade / Boas práticas / SEO	93 / 100 / 100	100 / 100 / 100

Sumário

1. Projeto 3
2. Metodologia de medição 3
3. Relatório "antes" 4
3.1 Medianas 4
3.2 Elemento LCP e fases 4
3.3 Requisições (Mobile) 4
3.4 Auditorias reprovadas 5
3.5 Gargalos identificados 5
4. Otimizações aplicadas 6
4.1 Fundo do hero sob demanda (G1) 6
4.2 Foto do Sobre (G2) 6
4.3 Fontes no próprio site (G3) 6
4.4 Code-split dos efeitos fora da primeira tela (G4) 6
4.5 Acessibilidade (G5, G6) 6
5. Relatório "depois" 7
5.1 Medianas 7
5.2 Fases do LCP (Mobile) 7
5.3 Requisições (Mobile) 7
6. Comparativo antes × depois 9
6.1 Mobile 9
6.2 Desktop 9
6.3 Impacto de cada etapa (Mobile) 9
7. Conclusão 10
Próximos passos 10
Anexos 10

(Se editar os títulos: botão direito no sumário → Atualizar campo.)

1. Projeto

O portfólio é uma SPA em **React 19 + Vite 8**, publicada no GitHub Pages. A identidade visual tem um tema de "sala de controle": o hero tem um fundo de terminal animado em **WebGL** (`FaultyTerminal`) e as seções usam animações do React Bits com **GSAP** e **Motion** (texto embaralhado, revelação ao rolar, letreiro, faixa curva e um cursor-mira).

O estado original foi congelado na tag `antes` do repositório. Cada otimização entrou num commit separado, e cada commit foi compilado e medido isoladamente. Assim dá para atribuir o ganho de cada etapa.

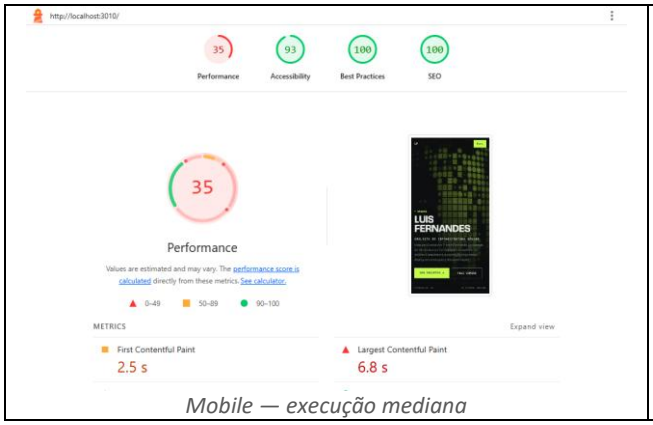
2. Metodologia de medição

Item	Valor
Ferramenta	Lighthouse 13.5.0 (CLI): Performance, Acessibilidade, Boas práticas e SEO
Navegador	Microsoft Edge 156 (Chromium), headless
Máquina	Intel Core i7-1255U, 16 GB RAM, Windows 11 Pro
Build	Cada commit extraído com `git archive` e compilado com `vite build` (produção, minificado)
Servidor	`serve` com gzip ligado, em localhost (como o GitHub Pages, que também comprime)
Execuções	3× Mobile (Moto G Power emulado, 4G lento e CPU 4× mais lenta) + 3× Desktop; valor reportado = mediana
Condições	Todas as etapas medidas na mesma sessão e com o mesmo script (`scripts/medir.mjs`)

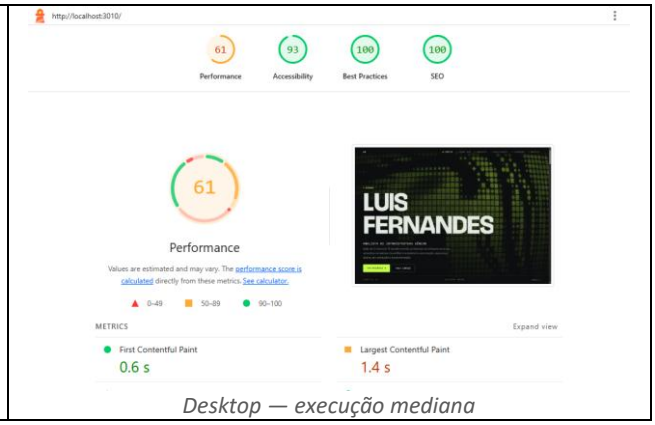
A mediana de 3 execuções reduz o efeito do ruído. No "depois", por exemplo, a nota Mobile variou entre 81 e 87 e o TBT entre 164 e 324 ms, sem nenhuma mudança no código.

Em modo headless o navegador desenha WebGL por software, sem placa de vídeo. Isso deixa o custo do fundo animado maior do que num celular real. Mesmo assim, o PageSpeed Insights do Google mede da mesma forma, então é esse custo que derruba a nota pública do site.

3. Relatório "antes"



Mobile — execução mediana



Desktop — execução mediana

3.1 Medianas

Métrica	Mobile	Desktop
Performance	35	61
Acessibilidade	93	93
Boas práticas	100	100
SEO	100	100
FCP	2,53 s	0,86 s
LCP	6,82 s	1,43 s
TBT	6481 ms	964 ms
CLS	0,000	0,000
Speed Index	8,38 s	2,44 s
Execução de JavaScript	12,88 s	2,56 s
Peso total	1830,6 KB	1831,8 KB
Requisições	11	12

3.2 Elemento LCP e fases

Nas 6 execuções, o elemento LCP foi o parágrafo de resumo do hero (`<p class="resumo">`), e não uma imagem. Abaixo estão as fases do LCP na execução mediana do Mobile, em tempos observados sem throttling: o texto chegava em 23 ms, mas levava mais de 3,8 s para ser desenhado, porque a thread principal estava ocupada com JavaScript.

Fase	Duração
Tempo até o primeiro byte (TTFB)	23 ms
Atraso de renderização	3821 ms

3.3 Requisições (Mobile)

Recurso	Tipo	Transferido	Prioridade
(documento HTML)	Document	1,2 KB	VeryHigh
fonts.googleapis.com/…/css2	Stylesheet	1,3 KB	VeryHigh
assets/index-CzzTXY34.js	Script	200,3 KB	High

Recurso	Tipo	Transferido	Prioridade
assets/index-CJd1A_Ry.css	Stylesheet	5,5 KB	VeryHigh
fonts.gstatic.com/…/3y9K6as8bTXq…NJ.woff2	Font	75,1 KB	VeryHigh
fonts.gstatic.com/…/tDbv2o-flEEn…-4.woff2	Font	30,6 KB	VeryHigh
assets/profile-pic-noback-270x338-BEXEpBEH.png	Image	1512,3 KB	Low
assets/loki-DucbXBFU.svg	Image	1,3 KB	Low
assets/aiops-CXMR3Tlj.svg	Image	1,3 KB	Low
favicon.svg	Other	1,7 KB	High

A foto do Sobre sozinha tinha 1,5 MB, ou seja, 83% dos bytes da página. As fontes vinham do Google Fonts: um CSS externo, que bloqueava a renderização, e dois arquivos de fonte em outro domínio.

3.4 Auditorias reprovadas

- **Total Blocking Time (6,5 s)**: a maior perda de pontos. Uma única tarefa longa, de mais de 5 s, montava o WebGL do hero.
- **Largest Contentful Paint (6,8 s)** e **Speed Index (8,2 s)**: consequência da mesma tarefa longa, que atrasava o primeiro desenho do texto.
- **Reduce JavaScript execution time (~13 s)** e **Minimize main-thread work (~17 s)**, na execução mediana.
- **Improve image delivery**: economia estimada de 1.486 KiB, por causa do PNG de 1121×1403 exibido com no máximo 360 px.
- **Render-blocking requests**: o CSS do Google Fonts bloqueava a renderização (~600 ms estimados).
- **aria-prohibited-attr**: `aria-label` em `<span>` sem `role` (nome animado do h1).
- **color-contrast**: as palavras do texto "Sobre" começavam com 12% de opacidade (contraste de 1,27:1).

3.5 Gargalos identificados

#	Gargalo (evidência)	Métricas	Impacto
G1	Fundo do hero em WebGL montado no carregamento: uma tarefa de mais de 5 s na thread principal, no bundle principal	TBT, LCP, SI	Crítico
G2	Foto do Sobre em PNG 1121×1403 (1,5 MB), exibida com no máximo 360 px, sem width/height	Peso, SI	Alto
G3	Fontes do Google Fonts: CSS externo bloqueante e 2 fontes em outro domínio (~107 KB)	FCP, LCP	Médio
G4	Letreiro, faixa curva e cursor-mira no bundle principal, embora fiquem abaixo da dobra ou não sirvam no celular	JS, TBT	Baixo
G5	`aria-label` em `span` sem role e nome acessível do logo diferente do texto visível	Acessibilidade	Médio
G6	Texto do Sobre começando com 12% de opacidade (contraste 1,27:1)	Acessibilidade	Médio

4. Otimizações aplicadas

4.1 Fundo do hero sob demanda (G1)

- O `FaultyTerminal` virou um chunk separado (`React.lazy`) e só é montado na **primeira interação** do visitante (mouse, toque, rolagem ou teclado), pelo hook `useAposInteracao` (com teste no Vitest).
- Até lá o hero mostra `hero-poster.webp` (34 KB), uma imagem estática da mesma grade. Quando o WebGL entra, a imagem some com uma transição, sem salto visual.
- Resultado da etapa: TBT Mobile de 6.481 ms para 130 ms. Foi a otimização de maior impacto do projeto.

4.2 Foto do Sobre (G2)

- PNG convertido para WebP em 540×676 (2× o tamanho exibido), com `width`/`height` para reservar o espaço e `loading="lazy"`, já que fica abaixo da dobra.

Imagem	Antes	Depois
Foto do Sobre	PNG 1121×1403 — 1.512 KB	WebP 540×676 — 38 KB (−97%)
Fundo do hero (1ª tela)	WebGL renderizado no carregamento	WebP 34 KB; WebGL só após interação

4.3 Fontes no próprio site (G3)

- Google Fonts trocado por **Fontsource** (`@fontsource-variable/bricolage-grotesque` com eixo de tamanho óptico e `@fontsource-variable/jetbrains-mono`), empacotado pelo Vite no mesmo domínio do site.
- `<link rel="preload">` da fonte dos títulos, que aparece na primeira tela. O Vite reescreve o caminho para o arquivo final com hash.
- O CSS externo bloqueante e as conexões com dois domínios do Google deixaram de existir. O `unicode-range` faz o navegador baixar só o subconjunto latino.

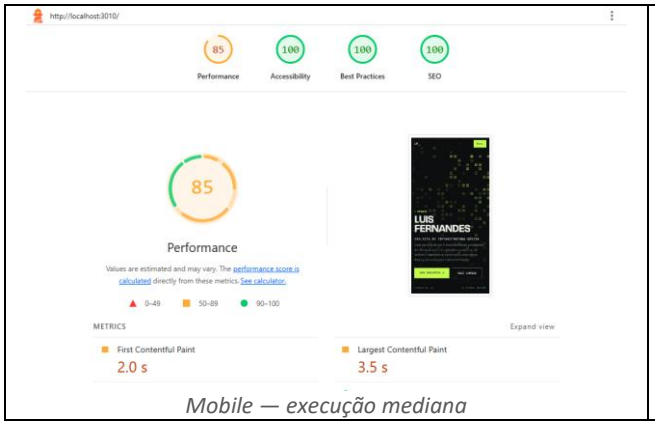
4.4 Code-split dos efeitos fora da primeira tela (G4)

- `ScrollVelocity` (letreiro), `CurvedLoop` (faixa curva) e `TargetCursor` (cursor-mira) passaram a ser carregados com `React.lazy` + `Suspense`.
- O cursor-mira só é baixado quando o dispositivo tem mouse (`(hover: hover) and (pointer: fine)`). No celular ele nem entra na lista de requisições.

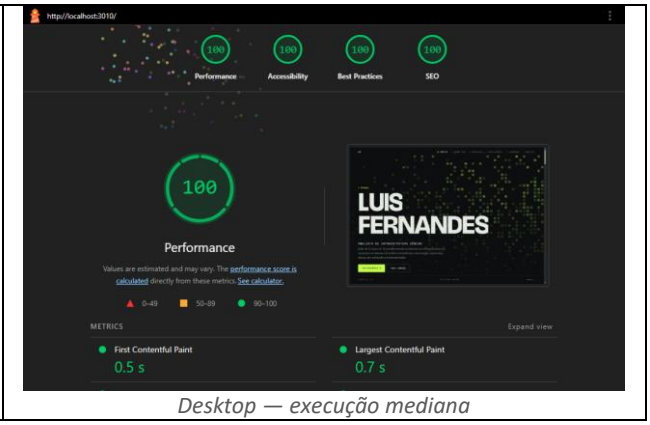
4.5 Acessibilidade (G5, G6)

- Nome do `h1` lido por um `span.sr-only`. As letras animadas ficam com `aria-hidden`, e o `Shuffle` deixou de pôr `aria-label` em `span`.
- O nome acessível do logo agora começa pelo texto visível "LF".
- O texto do Sobre surge do zero (`baseOpacity={0}`) em vez de começar meio apagado. O efeito de revelar ao rolar foi mantido.

5. Relatório "depois"



Mobile — execução mediana



Desktop — execução mediana

5.1 Medianas

Métrica	Mobile	Desktop
Performance	85	100
Acessibilidade	100	100
Boas práticas	100	100
SEO	100	100
FCP	2,03 s	0,49 s
LCP	3,54 s	0,75 s
TBT	211 ms	17 ms
CLS	0,000	0,000
Speed Index	2,03 s	0,52 s
Execução de JavaScript	1,18 s	0,27 s
Peso total	396,1 KB	401,3 KB
Requisições	15	18

5.2 Fases do LCP (Mobile)

Fase	Duração
Tempo até o primeiro byte (TTFB)	9 ms
Atraso de renderização	333 ms

O elemento LCP continua sendo o parágrafo de resumo do hero, mas o atraso de renderização observado caiu de 3.821 ms para 333 ms. O LCP simulado de 3,5 s vem da cadeia de rede do Mobile emulado (4G lento): como o texto é desenhado pelo React, ele depende do download do bundle principal (184 KB comprimido).

5.3 Requisições (Mobile)

Recurso	Tipo	Transferido	Prioridade
(documento HTML)	Document	1,2 KB	VeryHigh
assets/bricolage-grotesque-latin-opsz-normal-Cre6nC2_.woff2	Font	75,4 KB	High

Recurso	Tipo	Transferido	Prioridade
assets/index-Cpx3l7_Z.js	Script	184,0 KB	High
assets/index-DSFn9WKr.css	Stylesheet	8,0 KB	VeryHigh
assets/ScrollVelocity-Bw_a7-Bx.js	Script	7,1 KB	High
assets/ScrollVelocity-C-6PR3f7.css	Stylesheet	0,6 KB	VeryHigh
assets/CurvedLoop-5cGsInLM.js	Script	3,5 KB	High
assets/CurvedLoop-BcWzG1NS.css	Stylesheet	0,6 KB	VeryHigh
assets/hero-poster-DxnjO1_a.webp	Image	33,9 KB	High
assets/jetbrains-mono-latin-wght-normal-B9CIFXIH.woff2	Font	39,8 KB	VeryHigh
assets/foto-perfil-tKIsezCZ.webp	Image	37,7 KB	Low
assets/loki-DucbXBFU.svg	Image	1,3 KB	Low
assets/aiops-CXMR3Tlj.svg	Image	1,3 KB	Low
favicon.svg	Other	1,7 KB	High

Agora não há nenhum recurso de outro domínio. O WebGL (`FaultyTerminal`) e o cursor-mira não aparecem na lista, porque só seriam baixados após uma interação ou num dispositivo com mouse. O letreiro e a faixa curva viraram arquivos próprios, mas ainda são pedidos logo no carregamento, pois o React renderiza a página inteira de uma vez.

6. Comparativo antes × depois

6.1 Mobile

Métrica	Antes	Depois	Variação
Performance	35	85	+50
Acessibilidade	93	100	+7
Boas práticas	100	100	+0
SEO	100	100	+0
FCP	2,53 s	2,03 s	−20%
LCP	6,82 s	3,54 s	−48%
TBT	6481 ms	211 ms	−6270 ms
CLS	0,000	0,000	=
Speed Index	8,38 s	2,03 s	−76%
Execução de JavaScript	12,88 s	1,18 s	−91%
Peso total	1830,6 KB	396,1 KB	−78%
Requisições	11	15	+4

6.2 Desktop

Métrica	Antes	Depois	Variação
Performance	61	100	+39
FCP	0,86 s	0,49 s	−43%
LCP	1,43 s	0,75 s	−48%
TBT	964 ms	17 ms	−947 ms
CLS	0,000	0,000	=
Speed Index	2,44 s	0,52 s	−79%
Execução de JavaScript	2,56 s	0,27 s	−89%
Peso total	1831,8 KB	401,3 KB	−78%
Requisições	12	18	+6

6.3 Impacto de cada etapa (Mobile)

Etapa	Score	TBT	LCP	Peso
Antes	35	6481 ms	6,82 s	1830,6 KB
+ WebGL adiado, foto WebP e a11y do hero	83	130 ms	3,82 s	383,5 KB
+ Code-split (letreiro, faixa, cursor)	75	265 ms	4,04 s	385,1 KB
+ Fontes no próprio site + preload	81	308 ms	3,62 s	396,1 KB
Final (+ contraste do Sobre)	85	211 ms	3,54 s	396,1 KB

A etapa do hero concentra quase toda a melhoria: só ela levou a nota Mobile de 35 para 83 e o TBT de 6,5 s para 130 ms. O peso caiu de 1,8 MB para 384 KB, graças à foto em WebP.

O code-split não trouxe ganho mensurável. A queda de 83 para 75 e o TBT de 265 ms ficam dentro do ruído entre execuções, porque GSAP e Motion continuam no bundle principal (são usados no hero e no Sobre) e os chunks separados são pedidos no carregamento mesmo assim.

As fontes no próprio site reduziram o LCP Mobile (4,04 s → 3,62 s) e levaram o Desktop a 100. O peso subiu 11 KB porque o JetBrains Mono variável cobre todos os pesos num arquivo só; em troca, não há mais CSS externo bloqueando.

A correção de contraste levou Acessibilidade a 100. A variação de nota e de TBT nessa última etapa também é ruído: o commit só mudou um parâmetro de opacidade.

7. Conclusão

- O maior gargalo não era peso de arquivo, e sim **trabalho na thread principal**: montar o WebGL no carregamento travava a página por mais de 5 s no celular emulado. Adiar o efeito para a primeira interação, mostrando antes uma imagem leve da mesma grade, manteve a identidade visual e tirou mais de 6 s de TBT.
- A foto em PNG era 83% dos bytes da página. Redimensionar para 2× o tamanho exibido e converter para WebP reduziu o peso total em 78%.
- Hospedar as fontes no próprio site e corrigir os pontos de acessibilidade levaram Acessibilidade, Boas práticas e SEO a 100 no Mobile e no Desktop. No Desktop a Performance também chegou a 100.
- O code-split, sozinho, não move a nota quando as bibliotecas pesadas continuam no bundle principal. Ficou registrado como aprendizado.

Próximos passos

- **Pré-renderizar o HTML do hero** no build (SSG). Hoje o LCP depende do download e da execução do bundle de 184 KB; com o texto já no HTML, ele seria desenhado antes do JavaScript, o caminho para passar de 90 no Mobile.
- Carregar letreiro e faixa curva só quando entram na tela (`IntersectionObserver`), e avaliar trocar o `Shuffle` do hero por uma animação em CSS, para tirar GSAP e SplitText do bundle inicial.
- Cache longo (`immutable`) para `/assets/`: o GitHub Pages fixa `max-age=600`. Exigiria uma CDN na frente (ex.: Cloudflare) ou outro host.
- Rodar o Lighthouse CI no GitHub Actions com orçamento de performance, para nenhuma mudança futura trazer o problema de volta, e conferir os dados de campo no PageSpeed Insights depois do deploy.

Anexos

Os relatórios completos do Lighthouse (HTML de cada execução) e as medianas estão no repositório:

- `docs/relatorios/antes/`: `antes-*.report.html`, capturas e `resumo.md`
- `docs/relatorios/depois/`: `depois-*.report.html`, capturas e `resumo.md`
- `docs/relatorios/etapas/`: medianas após cada etapa de otimização
- `scripts/medir.mjs`: script que mede, calcula as medianas e gera os resumos